

PCA and Clustering

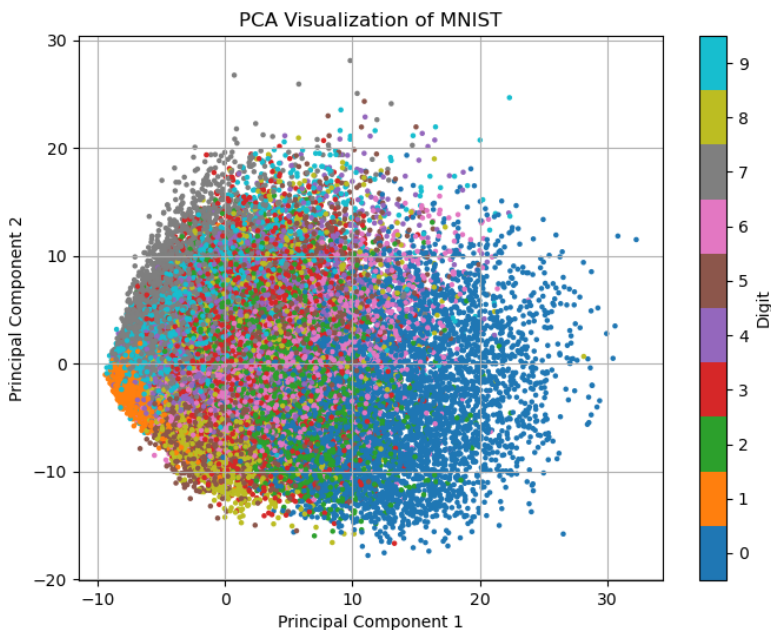
Programming Questions

1 Question 1. Dimensionality reduction and visualization

For this programming section, we will use the MNIST1 . As we discussed before, the dataset comprising images of handwritten digits, centered and resized to 28×28 pixels. Each image has been saved as a vector with the dimension of $784 = 28 \times 28$, and is associated with a label, namely, digits 0 through 9. We also used previously introduced machine learning library for python programming language, scikit-learn, often referred to as sklearn. Please complete the code based on the text provided for each section in the Python file. The blank sections that need completion are indicated by XXX. The completed python files do not need to be submitted.

1.1 Question 1.1

As we discussed, each digit in the dataset can be represented as a 784-dimensional vector, but visualizing data in 784 dimensions is not practically feasible. Here, we aim to reduce this to 2 dimensions for visualization purposes. We will utilize the first section of the PCA.py script to accomplish this task. After filling in the necessary sections of the script, you should be able to generate a 2D plot of the data. Upon examining the plot, discuss whether we can identify distinct clusters within the data. Do you expect different runs of applying PCA results in the same output? Why?

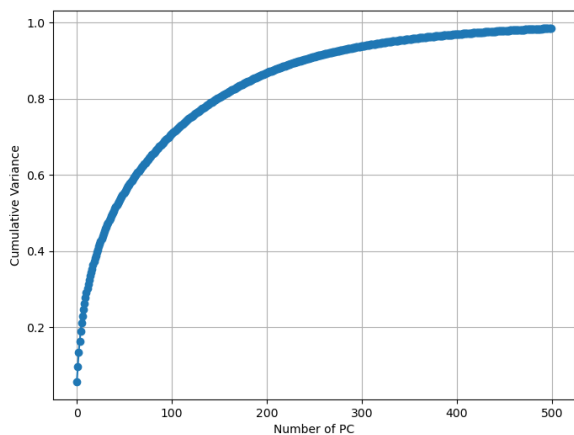


After applying PCA dimensionality reduction, we can observe data points clustered into different clusters in the generated 2D graphs, which represent different handwritten digits(blue cluster, red cluster, yellow cluster, etc.). Since PCA tries to preserve maximum variance, similar numbers will tend to cluster together in the two main components.

We can expect different runs of applying PCA results in the same output, since according to PCA.py, the PCA algorithm is always initialized the same, using the same method to select the initial principal component and the data set does not change, therefore the result should be the same every time the PCA is runned.

1.2 Question 1.2

In the second section of the PCA.py script, we will plot the cumulative variance explained by adding more principal components. Determine the number of components needed to capture 90% of the variance in the data.

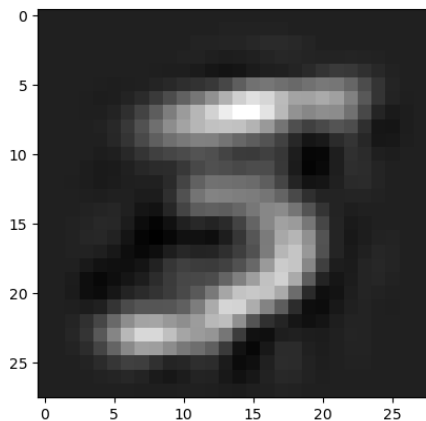


According to the generated graph, we can see that the number of components needed to capture 9/10 of the variance in the data is approximately 245.

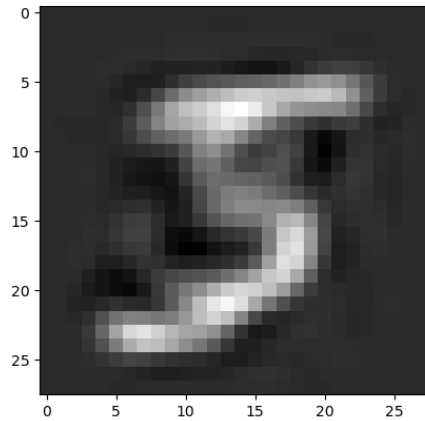
1.3 Question 1.3

In this section, we will use the third section of the PCA.py script to reconstruct input images using the first 50,100, and 500 principal components. Visualize the output and discuss the properties of added components?

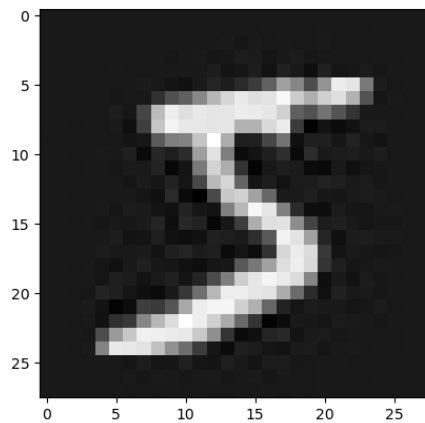
Component = 50:



Component = 100:



Component = 500:

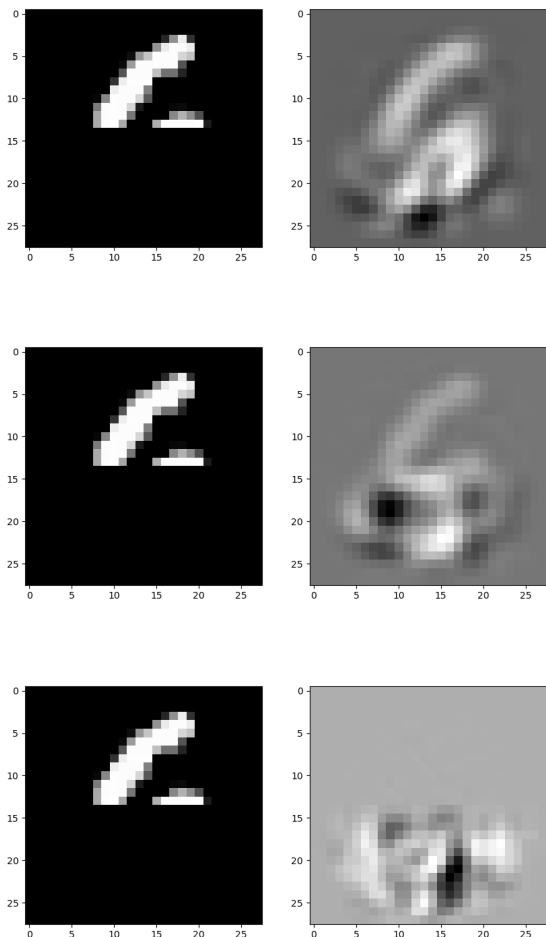


Properties of the added components: When component = 50, the result image presents a blurry picture because only a small fraction of the dimension of the data is captured. At this point, the image is a rough representation of the shape of the number, with less detail, showing the initial effects of the reconstruction process; when component = 100, the clarity of the image is improved compared to using only 50 principal components, since more data is captured, making the contours and some details of the numbers (such as the "l" within 5 start to become more legible); when component = 500, the image appears sharper and richer in detail. Since most of the data is included, the reconstructed image is more accurately reflect the features of the original image. The shape, edges and internal details of the number are restored.

We can see that as the value of component increases, the result image becomes clearer and can distinguish the "5" written by hand. In PCA, principal components are extracted from the original data set to capture the main features of the original data. Therefore the more components added, the less the "lost data", and the image is more distinguishable.

1.4 Question 1.4

As discussed in theoretical question 2, PCA can be employed to address missing data. In this section, we introduce missing data by removing the bottom half of an image shown in Figure 2 Using the mathematics derived from 2.b, complete the fourth section of the PCA.py script to reconstruct the input image. Visualize the output and discuss how increasing the number of principal components changes the reconstructed image.



As we used 50, 100, and 500 principal components to reconstruct an image of a handwritten number "6" with the lower half removed, the results showed that the reconstructed image became more chaotic as the number of principal components increased. When component = 50, the reconstructed picture can roughly be distinguished as 6, but the circle in 6 became bigger when component = 100, and the result image become undistinguishable when component = 500.

When we introduce more principle component to be counted, the model not only captures major variances in the data but also starts to capture unimportant noise (messy, cluttered pixels within original picture) and mess up with the development of the correct model. Also when the component number grows too big, we might encounter overfitting and the model is attempting to reconstruct the number while also misinterpreting and reconstructing noise in the data, resulting in the reconstructed image looking more cluttered.

2 Question 2. Segmentation with Clustering

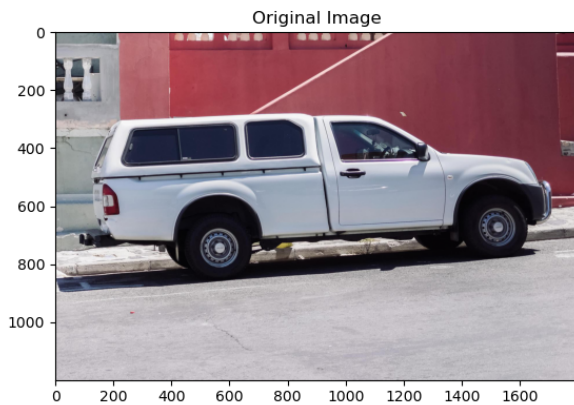
Image segmentation stands as a fundamental task in the computer vision, crucial for pinpointing objects and delineating boundaries within images. Essentially, image segmentation is the assignment of a label to each pixel in an image, grouping pixels with similar attributes together. This process finds extensive application in everyday scenarios like autonomous vehicles, facial recognition and detection, video surveillance, and satellite image analysis. A multitude of algorithms can be enlisted to accomplish the segmentation of regions of interest within input data. Previously, a simple approach involved leveraging clustering algorithms atop RGB images. This method entailed grouping pixels based on their color similarity, allowing for the identification and isolation of distinct regions within the image. In this question, we are looking to explore how to use KMeans and GMM clustering algorithm to segment an input image. We will use sklearn to perform clustering. Having this in mind, please answer below questions.

2.1 Question 2.1

We will use the first section in Clustering.py to explore the argument for the KMeans algorithm. We are looking to segment an input image into 3 segments. Execute the algorithm and display the segmented image. Report the cluster center. If we run the algorithm three times, will we obtain consistent results? (Note: you should compare

the center of the cluster). Will we consistently receive the same labels each time we run the model? Discuss your observation.

The original picture is:



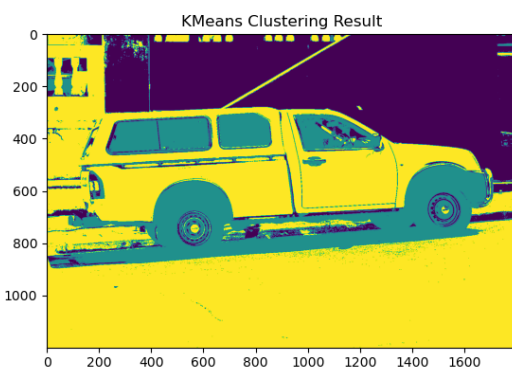
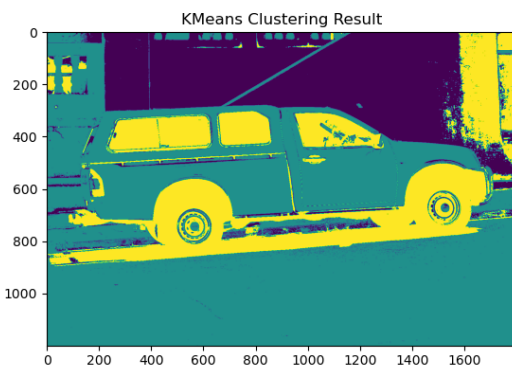
By using `print(kmeans.cluster_centers_)`, I get the cluster centers for each of executions during 3 run-time:

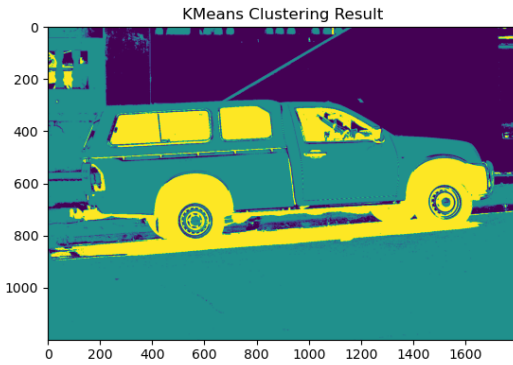
1st cluster centers: `[[0.50201315 0.25841242 0.30504925] [0.0883945 0.08545425 0.13049737] [0.73330868 0.72483125 0.74977238]]`

2nd cluster centers: `[[0.1696647 0.11370172 0.1608789 ] [0.73411347 0.72750682 0.75244757] [0.53911114 0.28225914 0.3269089 ]]`

3rd cluster centers: `[[0.50201315 0.25841242 0.30504925] [0.0883945 0.08545425 0.13049737] [0.73330868 0.72483125 0.74977238]]`

the result image:

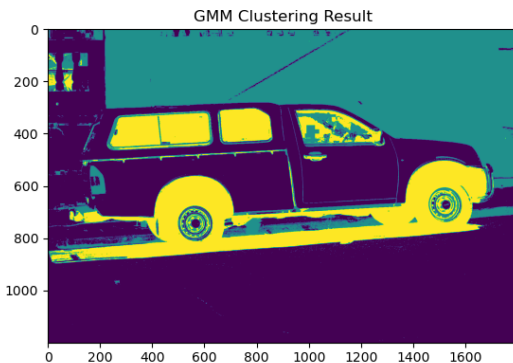




If we run the algorithm three times, because we randomly selecting the initial cluster center of the KMeans algorithm during initialization, the cluster center may be different each time the algorithm is run, thus even with the same input data, different runs may result in different clustering results. As shown in the result, we may obtain similar output in 1st and 3rd run, but the 2nd result shown in graph is entirely different from that we get in 1st and 3rd run.

2.2 Question 2.2

We will use the second section in Clusteirng.py to explore the argument for the GMM algorithm. We are interested to explore how full covariance shape to segment the input image into 3 segments. Show the results of the segmentation. Report the mean and covariance of each component.



by using `print(gmm.means_)` and `print(gmm.covariances_)`, we have:

Means (Cluster Centers) of GMM:

```
[[0.72959546 0.72817597 0.75420619]
 [0.51249323 0.27081747 0.31409667]
 [0.08539084 0.08397403 0.12992251]]
```

Covariances of GMM:

```
[[[0.01034863 0.00862341 0.00748354]
 [0.00862341 0.00759759 0.00689717]
 [0.00748354 0.00689717 0.00677471]]
```

```
[[0.01981011 0.00837886 0.0075096 ]
 [0.00837886 0.01155624 0.00989568]
 [0.0075096 0.00989568 0.00903889]]
```

```
[[0.00191286 0.00212362 0.00317897]
 [0.00212362 0.00254957 0.00398801]
 [0.00317897 0.00398801 0.00665537]]]
```